

Assignment Task — Build a Full-Stack AI-Powered Hotel Guest Assistant

Scenario

A hotel wants to add an AI-powered guest assistant to its website. Guests should be able to ask questions about the property and check room availability through a simple web interface.

Typical questions may include:

- What time is check-in?
- Does the hotel have a swimming pool?
- Which room is suitable for three guests?
- Is breakfast included?
- What is the cancellation policy?
- Do you have rooms available for a given date?

Build a small full-stack application that provides the guest-facing experience and the backend/AI service required to answer these questions.

What You Need to Build

Frontend

Build a simple guest-facing web application. React / Next.js is preferred, but an equivalent modern frontend framework is acceptable.

The frontend should:

- Provide a clean chat or conversational interface.
- Allow a guest to enter and submit questions.
- Display guest messages and assistant responses clearly.
- Show loading/processing states while waiting for a response.
- Handle and display API or AI failure states gracefully.
- Support follow-up questions in the same conversation.
- Provide a usable way to collect check-in date, check-out date, and number of guests when availability is requested.
- Display availability results in a clear, user-friendly way.
- Be reasonably responsive for desktop and mobile screens.
- Call the backend service rather than calling the LLM directly from the browser.
- Avoid exposing API keys or sensitive configuration in frontend code.

Backend

Build the backend service and APIs used by the frontend.

The backend should:

- Expose an API that accepts guest questions and conversation context.
- Use a small hotel knowledge base provided as JSON or stored in a database.
- Answer property, room, amenity, policy, and FAQ-related questions from the provided data.
- Identify when a guest is asking about room availability.
- For availability requests, call a mock function/tool such as `checkAvailability(checkIn, checkOut, adults)`.
- Use an LLM or AI model where appropriate.
- Keep deterministic business logic outside the LLM where appropriate.
- Return a clear fallback response when an answer cannot be reliably determined from the available data.
- Maintain basic conversation context where useful.
- Return clean, structured API responses that are easy for the frontend to consume.
- Include basic validation, error handling, and logging.
- Include meaningful automated tests for important backend flows.

Frontend + Backend Integration

The frontend and backend should work together as one usable flow.

We should be able to:

- Open the web application.
- Ask a hotel-related question.
- See the frontend call the backend.
- Receive and display an AI response.
- Ask a follow-up question.
- Check room availability.
- See a useful error or fallback when something fails.

AI + Product Thinking Expectations

Along with the implementation, explain:

- What customer problem are you solving?
- What does the guest journey look like?
- Why did you design the frontend experience the way you did?
- Which parts should use AI and which parts should remain deterministic?
- What can go wrong with the AI response?
- How would you prevent hallucinations or unsupported answers?
- What should happen when the model, frontend API call, or another dependency fails?

- How would you measure whether the feature is actually useful?
- What would you improve before taking this to production?

Evaluation & Testing

Create at least 8–10 meaningful test/evaluation scenarios covering:

- Normal guest questions.
- Questions with missing information.
- Ambiguous questions.
- Availability/tool-calling requests.
- Incorrect or unsupported assumptions.
- Conversation follow-ups.
- Frontend loading and error states.
- Backend or model failure/fallback scenarios.
- At least one end-to-end frontend-to-backend flow.

Deliverables

- GitHub repository containing both frontend and backend source code.
- README with setup and run instructions for the complete application.
- Short architecture explanation covering frontend, backend, AI/model, and data flow.
- A working frontend that can be run locally.
- Backend API examples using curl, Postman, or equivalent.
- A short note covering product, UX, engineering, and AI decisions.
- Evaluation/test scenarios and observed results.
- List of AI tools used during development, if any.
- Optional: a deployed demo URL or a short screen recording of the working application.

AI Tools

You may use tools such as ChatGPT, Claude, Cursor, Codex, or similar tools while completing the assignment. However, you should be able to explain and defend every important technical, frontend, backend, and product decision in your submission.

Time Guidance

Target approximately 6–8 hours of focused effort. We are not looking for a production-perfect application. We want to understand how you prioritize, think, design, build, integrate, test, and communicate.